TRAFFIC ACCIDENT SEVERITY PREDICTION

Project Id:20

Name : Anshika Gaur (Team Leader)

Class Roll Number: 1

University Roll Number : 2027616

Section and Group: G(G1)

Name : Anushka Jain

Class Roll Number: 33

University Roll Number : 2028655

Section and Group: G(G1)

TABLE OF CONTENTS

1. **Problem Description** 3
   - 1.1. **Dataset Description** 3
   - 1.2. **Flow diagram** 4
2. **Modules for Project Implementation** 5
   - 2.1. **Modules Used** 5
   - 2.2. **Platform Used** 6
3. **Machine Learning model/ Searching Technique used** 7
4. **Evaluation metrics used** 8
5. **Results and Visualizations** 9
6. **Conclusion and Future Scope** 11
7. **References** 12

1. Problem Description

The **Traffic Accident Severity Prediction System** is a machine learning-based application developed to predict the severity of road accidents using factors such as speed, weather conditions, road conditions, and vehicle type. The system aims to assist in identifying high-risk scenarios and improving road safety through data-driven insights.

The dataset is preprocessed by handling missing values and removing irrelevant records to improve data quality. Feature selection techniques are applied to retain the most important attributes, such as speed and weather conditions, which significantly influence accident severity.

1.1 Dataset Description

The dataset used in the Traffic Accident Severity Prediction System contains information related to road accidents and the factors influencing their severity. It is used to train machine learning models to classify the severity level of accidents.

The dataset includes multiple features such as:

- **Speed of Vehicle:** Indicates how fast the vehicle was moving at the time of the accident
- **Weather Conditions:** Includes conditions like clear, rainy, foggy, etc.
- **Road Conditions:** Information about road surface (dry, wet, damaged)
- **Vehicle Type:** Type of vehicle involved (car, bike, truck, etc.)
- **Other Factors:** Time of accident, location, and traffic conditions (if available)

The dataset is stored in a structured format (CSV file) and is loaded using Python libraries like Pandas for further processing.

Data Preprocessing Steps:

To ensure accuracy and efficiency of the model, the dataset undergoes the following preprocessing steps:

- Handling missing or null values
- Removing duplicate and irrelevant records
- Encoding categorical variables (e.g., weather, vehicle type) into numerical form
- Feature selection to retain the most important attributes like speed and weather conditions
- Splitting data into training and testing sets

Target Variable:

The target variable represents the severity of the accident, which is classified into categories such as:

- Low Severity
- Medium Severity
- High Severity

Dataset Usage:

- Used to train multiple machine learning models such as Logistic Regression, SVM, KNN, and Random Forest
- Helps identify patterns and relationships between accident factors and severity
- Supports prediction of accident severity for new data inputs

This dataset plays a crucial role in building an intelligent system that can assist in improving road safety and reducing accident risks.

1.2 Flow Diagram

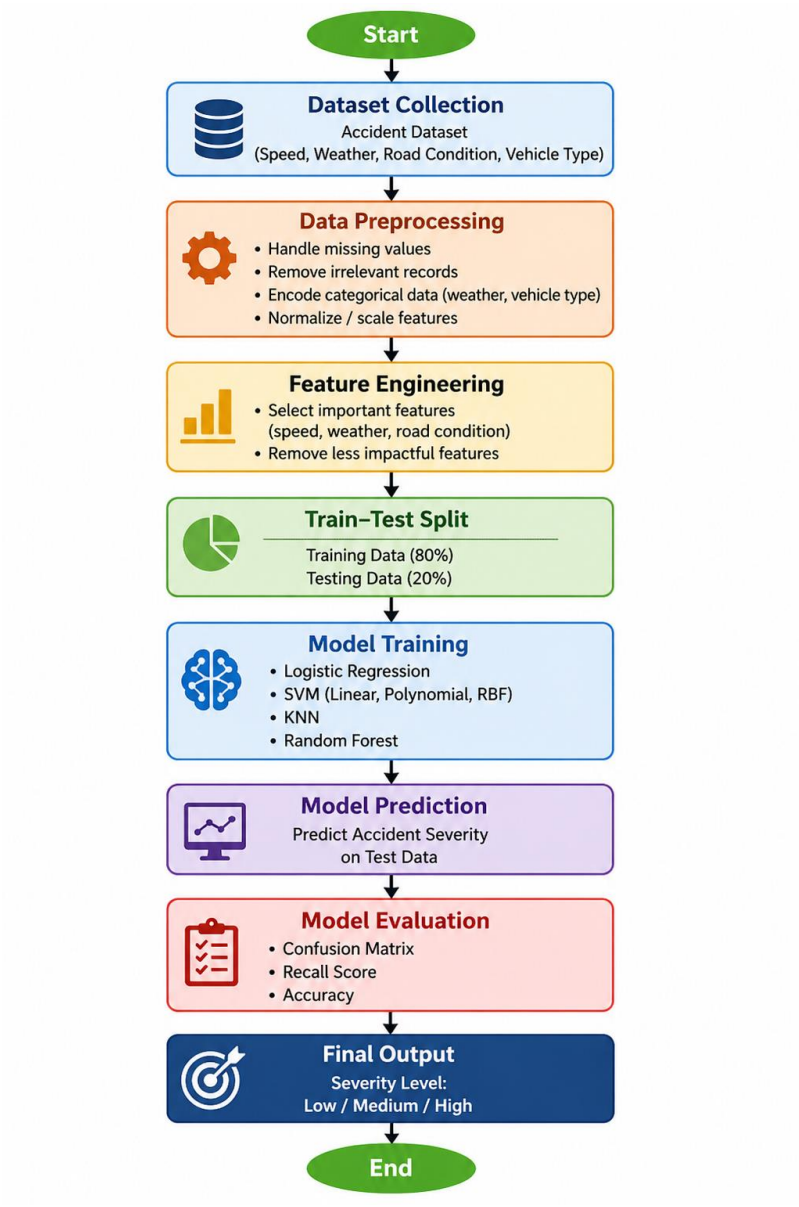


Fig.1 Sample flow diagram.

2. Modules for Project Implementation

2.1 Modules Used

The following inbuilt and external Python modules are used in the project for data processing, model building, and visualization:

1. Pandas

Pandas is used for data manipulation and analysis.

Functions used:

- `pd.read_csv()` – To load dataset from CSV file
- `df.head()` – To display first few rows of dataset
- `df.info()` – To get dataset information
- `df.drop()` – To remove unwanted columns
- `df.fillna()` – To handle missing values

2. NumPy

NumPy is used for numerical computations and handling arrays.

Functions used:

- `np.array()` – To create arrays
- `np.mean()` – To calculate mean values
- `np.std()` – To calculate standard deviation

3. Matplotlib

Matplotlib is used for data visualization through graphs.

Functions used:

- `plt.plot()` – To create line graphs
- `plt.scatter()` – To create scatter plots
- `plt.xlabel()` / `plt.ylabel()` – To label axes
- `plt.title()` – To add title to graph
- `plt.show()` – To display the graph

4. Seaborn

Seaborn is used for advanced and attractive statistical visualizations.

Functions used:

- `sns.heatmap()` – To display correlation heatmap
- `sns.countplot()` – To show frequency distribution

5. Scikit-learn (sklearn)

Scikit-learn is used for implementing machine learning models and evaluation.

a) Model Selection

- `train_test_split()` – To split dataset into training and testing sets

b) Algorithms Used

- `LogisticRegression()` – For classification
- `SVC()` – Support Vector Machine
- `KNeighborsClassifier()` – KNN algorithm
- `RandomForestClassifier()` – Random Forest model

c) Preprocessing

- `LabelEncoder()` – To convert categorical data into numerical form
- `StandardScaler()` – To normalize/scale data

d) Evaluation Metrics

- `accuracy_score()` – To calculate accuracy
- `classification_report()` – To display precision, recall, F1-score
- `confusion_matrix()` – To generate confusion matrix

2.2 Platform Used:

The following platforms are used for the development and implementation of the project:

1. Visual Studio Code

Used for writing, testing, and debugging the Python code.

VS Code provides a powerful development environment with features like syntax highlighting, code completion, and an inbuilt debugger. It also supports multiple extensions which enhance productivity and make coding more efficient.

2. Jupyter Notebook

Used for interactive coding, data analysis, and visualization.

Jupyter Notebook allows step-by-step execution of code, making it easier to test and understand machine learning models. It is especially useful for data preprocessing, visualization (graphs and plots), and experimenting with different algorithms. It also helps in documenting code with explanations and outputs in a single interface.

3. Python

Used as the main programming language for implementing the project.

Python is widely used in machine learning due to its simplicity and availability of powerful libraries such as Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn.

4. Operating System (Windows)

Used as the base platform to run all tools and software.

It provides compatibility with development tools like VS Code and Jupyter Notebook, ensuring smooth execution of the project.

3. Machine Learning Model/ Searching Technique Used

The project uses multiple machine learning algorithms to improve prediction accuracy and compare performance:

1. Logistic Regression
A supervised classification algorithm used for predicting categorical outcomes. It estimates the probability of a student falling into a performance category.

2. Support Vector Machine (SVM)
SVM is used with different kernels:

- **Linear Kernel** – for linearly separable data
- **Polynomial Kernel** – captures non-linear relationships
- **RBF Kernel** – handles complex patterns

It helps in finding the optimal hyperplane for classification.

3. K-Nearest Neighbors (KNN)
A simple algorithm that classifies a data point based on the majority class of its nearest neighbors. It is useful for pattern recognition.

4. Random Forest Classifier
An ensemble learning method that uses multiple decision trees. It improves accuracy and reduces overfitting by averaging predictions.

4. Evaluation metrics used

The performance of models is evaluated using the following metrics:

1. Accuracy
Measures overall correctness of the model.
Formula:
Accuracy = (TP + TN) / (TP + TN + FP + FN)

2. Precision
Measures how many predicted positives are actually correct.
Precision = TP / (TP + FP)

3. Recall (Sensitivity)
Measures how many actual positives are correctly identified.
Recall = TP / (TP + FN)

4. F1 Score
Harmonic mean of precision and recall.
F1 Score = 2 × (Precision × Recall) / (Precision + Recall)

5. Confusion Matrix
A table used to evaluate classification performance showing TP, TN, FP, FN.

5. Results and Visualizations

The system generates multiple visualizations to better understand data and model performance:

- **Heatmap**: Shows correlation between features
- **Confusion Matrix Visualization**: Shows classification performance

Comparison of models shows that:

- Random Forest gives high accuracy and stability
- SVM performs well with proper kernel selection
- KNN works well for smaller datasets
- Logistic Regression provides fast and interpretable results

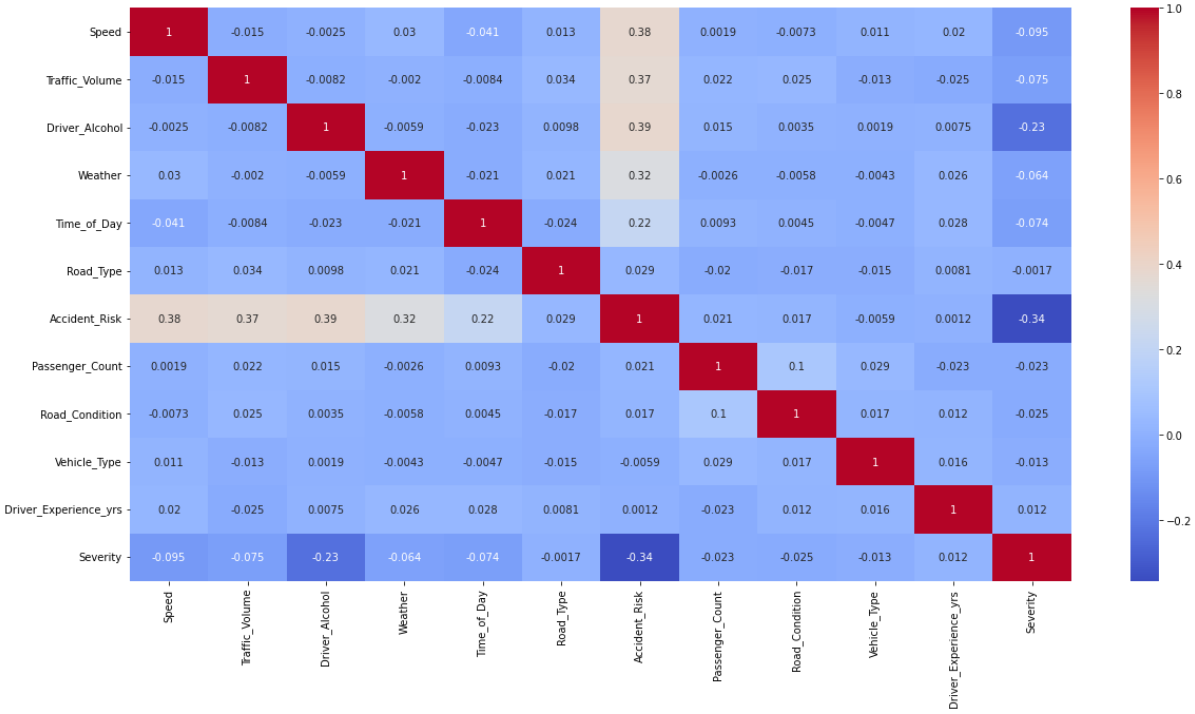
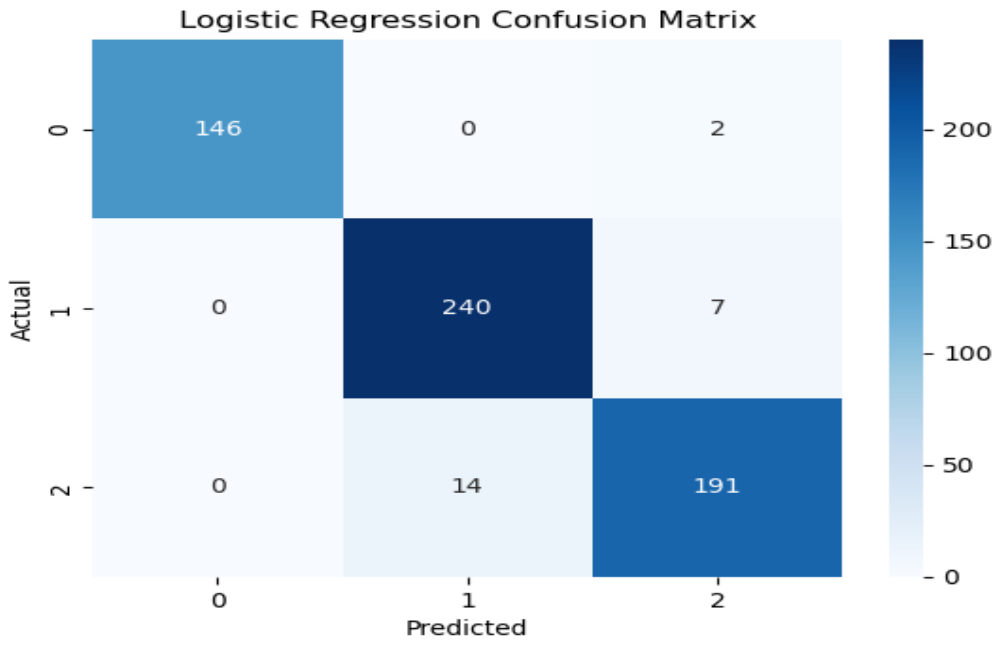
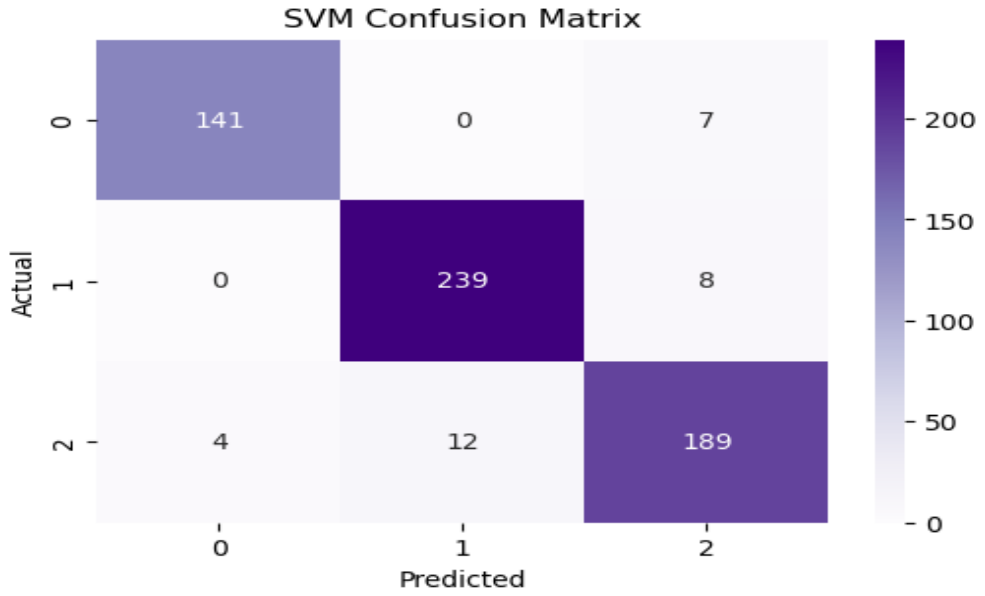


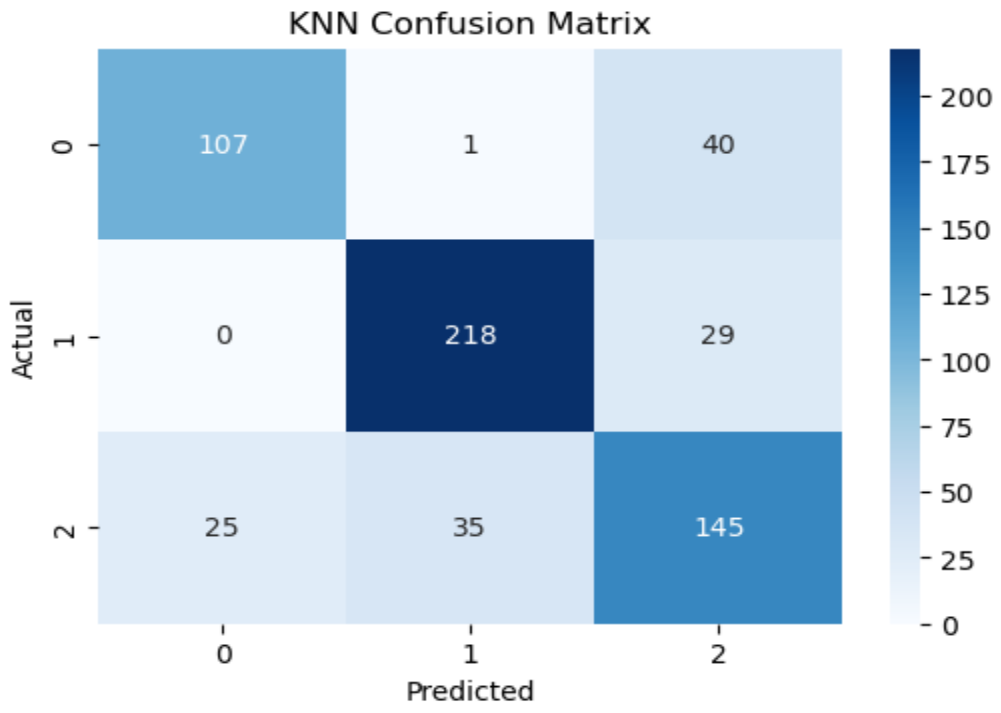
Fig.2 Heatmap showing corelation.

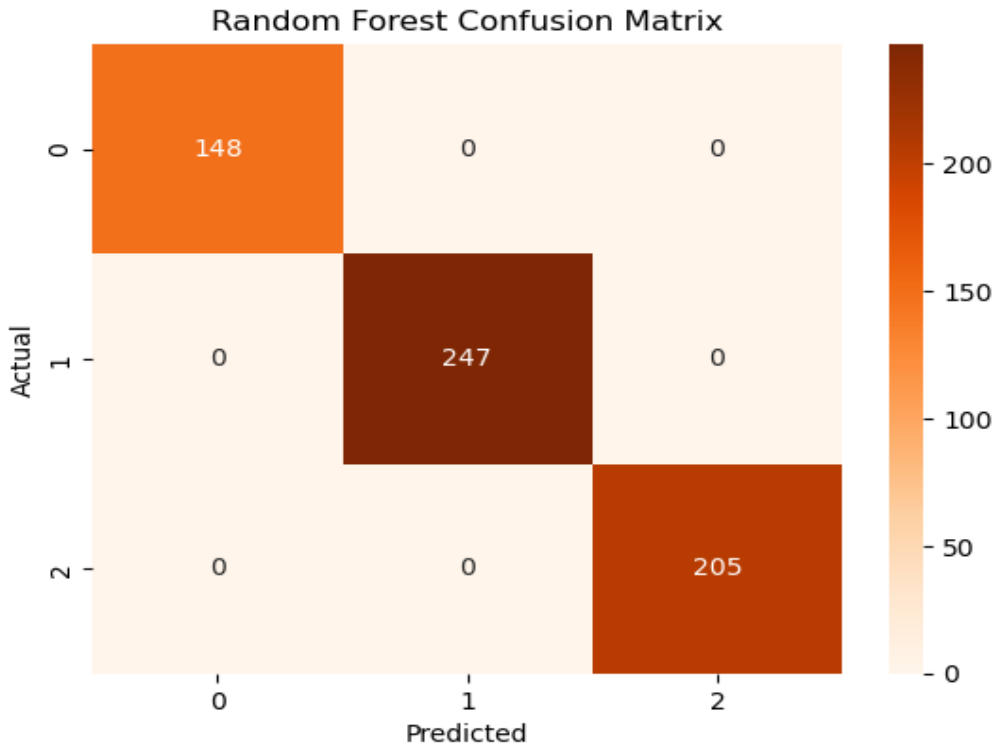
```
-- Logistic Regression --
              precision    recall  f1-score   support

           0       1.00      0.99      0.99       148
           1       0.94      0.97      0.96       247
           2       0.95      0.93      0.94       205
```

```
    accuracy                           0.96       600
   macro avg       0.97      0.96      0.96       600
weighted avg       0.96      0.96      0.96       600

Accuracy: 0.9616666666666667
F1 Score: 0.9616632142618267


-- SVM --
              precision    recall  f1-score   support

           0       0.97      0.95      0.96       148
           1       0.95      0.97      0.96       247
           2       0.93      0.92      0.92       205

    accuracy                           0.95       600
   macro avg       0.95      0.95      0.95       600
weighted avg       0.95      0.95      0.95       600

Accuracy: 0.9483333333333334
F1 Score: 0.9483101833024218


-- KNN --
             precision    recall  f1-score   support

          0       0.81      0.72      0.76       148
          1       0.86      0.88      0.87       247
          2       0.68      0.71      0.69       205

   accuracy                           0.78       600
  macro avg       0.78      0.77      0.78       600
weighted avg       0.78      0.78      0.78       600

Accuracy: 0.7833333333333333
F1 Score: 0.7832563650968322


=== Random Forest ===
             precision    recall  f1-score   support

          0       1.00      1.00      1.00       148
          1       1.00      1.00      1.00       247
          2       1.00      1.00      1.00       205

   accuracy                           1.00       600
  macro avg       1.00      1.00      1.00       600
weighted avg       1.00      1.00      1.00       600

Accuracy: 1.0
F1 Score: 1.0
```


Logistic Regression Confusion Matrix
146
0
2
0
240
7
0
14
191
Actual
0
1
2
Predicted
200
150
100
50
0


SVM Confusion Matrix
141
0
7
0
239
8
4
12
189
Actual
0
1
2
Predicted
200
150
100
50
0


KNN Confusion Matrix
107
1
40
0
218
29
25
35
145
Actual
0
1
2
Predicted
0
1
2
200
175
150
125
100
75
50
25
0


Random Forest Confusion Matrix
148
0
0
0
247
0
0
0
205
Actual
0
1
2
Predicted
0
1
2
200
150
100
50
0

6. Conclusion and Future Scope:

Conclusion:

The Student Performance Prediction System successfully predicts student outcomes using multiple machine learning models. It helps identify students at risk and supports data-driven decision-making. Among all models, Random Forest and SVM showed better performance in terms of accuracy.

Future Scope:

- Integration with real-time student databases
- Deployment as a web application
- Use of Deep Learning models for better accuracy
- Adding more features like attendance, behavior, etc.
- Visualization dashboard for teachers

7. References:

Books:

1. Let Us Python – Yashavant Kanetkar
2. Hands-On Machine Learning with Scikit-Learn – Aurélien Géron

Online Resources:

- GeeksforGeeks – https://www.geeksforgeeks.org/
- TutorialsPoint – https://www.tutorialspoint.com/
- Scikit-learn Documentation – https://scikit-learn.org/
- Kaggle – https://www.kaggle.com/
- Stack Overflow – https://stackoverflow.com